[정규표현식 핵심 개념 및 활용 보고서]

인공지능소프트웨어 2603110175 양담경

1. 개요

1) 정의
정규 표현식(Regular Expressions)은 문자열에서 특정한 패턴을 찾거나, 수정, 검증하기 위해 사용하는 문자열 검색 및 매칭 도구
2) 필요성
복잡한 조건문과 반복문 없이 몇 줄의 패턴식으로 문자열을 검증하고 변환할 수 있어 데이터 분석 및 텍스트 처리 과정의 효율을 극대화함. 파이썬(re 모듈)을 비롯한 대부분의 프로그래밍 언어에서 표준으로 지원함.

2. 파이썬 re 모듈 API 및 주요 메서드 체계

파이썬은 정규표현식을 지원하기 위해 표준 라이브러리인 re 모듈을 기본 제공함. 체계적이고 효율적인 패턴 분석을 위해서는 re 모듈이 제공하는 API 구조, 검색 메서드의 반환 타입, 컴파일 메커니즘을 명확하게 파악해야 함.

1) 패턴 컴파일 함수 분석

re.compile(pattern, flags=0)

전달받은 정규식 문자열 패턴을 파이썬 정규표현식 엔진이 해석할 수 있는 '패턴 객체(Pattern Object)'로 변환(컴파일)

동일한 정규표현식 패턴을 대용량 텍스트 처리 과정이나 반복문(for/while) 내부에서 재사용할 경우, 미리 `re.compile()`을 통해 패턴 객체로 변환해 두면 반복적인 패턴 해석 비용이 절감되어 실행 효율성 상승함.

2) 핵심 검색 및 치환 메서드 비교

re 모듈 및 컴파일된 패턴 객체는 탐색 범위와 목적에 맞춰 다음의 핵심 API 메서드 제공

findall()	문자열 전체	정규식 패턴과 매치되는 모든 문자열을 추출하여 리스트로 반환	매치된 문자열들의 리스트 (list)
finditer()	문자열 전체	정규식 패턴과 매치되는 모든 대상의 상세 정보를 순회 가능하도록 반환	Match 객체들의 이터레이터 (iterator)
match()	문자열 처음부터	문자열의 시작 부분이 정규식 패턴과 매치되는지 검사	Match 객체 또는 None
search()	문자열 전체	문자열 전체를 검색하여 정규식 패턴과 매치되는 첫 번째 위치 검사	Match 객체 또는 None
fullmatch()	문자열 전체	문자열 전체가 정규식 패턴과 정확히 일치하는지 검사	Match 객체 또는 None
split()	문자열 전체	정규식 패턴과 일치하는 부분을 기준으로 문자열을 분리	분리된 문자열들의 리스트 (list)
sub()	문자열 전체	정규식 패턴과 일치하는 부분을 지정한 다른 문자열 또는 치환 함수(Callable)의 반환값으로 치환	치환이 완료된 문자열 (str)

2) Match 객체의 세부 정보 추출 메서드

match()나 search() 호출 결과 매칭에 성공하면 Match 객체가 반환됨.

Match 객체는 위치 및 매치 텍스트 세부 정보를 조회하는 전용 메서드 제공함.

- group(): 매치된 실제 문자열 반환 (괄호 그룹 지정 시 그룹 번호 인자 전달 가능)
- start(): 매치된 문자열의 시작 인덱스 위치 반환

- end(): 매치된 문자열의 끝 인덱스 위치(미포함 경계) 반환
- span(): 매치된 문자열의 (시작 인덱스, 끝 인덱스) 튜플(tuple) 형태로 반환

3) 컴파일 옵션의 활용

패턴 매칭 시 조건 설정의 유연성 확보, 복잡한 정규식의 가독성을 높이기 위해

re 모듈은 컴파일 플래그 옵션 제공함.

- re.DOTALL (또는 re.S): 메타 문자인 점(.)이 줄바꿈 문자(\n)를 포함한 모든 문자와 매치되도록 설정
- re.IGNORECASE (또는 re.I): 대소문자 구별 없이 정규식 매칭
- re.VERBOSE (또는 re.X): 정규 표현식 내의 공백문자와 주석(#)을 무시하도록 설정하여, 복잡한 정규식을 여러 줄로 나열하고 라인별 설명을 추가

re 모듈 API 체계를 파악한 후, 실무 코드 작성 시 작성자를 가장 혼란스럽게 만드는 역슬래시(\)와 파이썬 문자열 간의 이스케이프 충돌 메커니즘을 다룸.

3. 파이썬 정규표현식의 핵심 작성 규칙

< 역슬래시()와 Raw String (r'...') >

파이썬에서 정규표현식을 작성할 때 입문자들이 가장 겪기 쉬운 오작동 문제는

역슬래시(\) 문자의 이중 해석 메커니즘에서 비롯됨.

1) 기본 메타 문자

문자	의미
.	줄바꿈을 제외한 임의의 한 문자
^ / $	문자열의 시작 / 끝
*	앞 문자 0회 이상 반복
+	앞 문자 1회 이상 반복
?	앞 문자 0회 또는 1회 (선택적)
{n,m}	n회 이상 m회 이하 반복
[ ]	문자 클래스 ([abc]), [^abc]는 부정
|	OR (또는)
( )	그룹 지정 (group(1)로 조회)
(?P<name>...)	이름 있는 그룹 (group('name')으로 조회)
\.	메타 문자를 문자 그대로 쓰기 위한 이스케이프

2) 메타 문자에서의 역슬래시() 단축 표현 기능

정규표현식에서 역슬래시는 특수 문자와 결합하여 자주 사용되는 문자 클래스를 직관적으로 표현하는 단축 기능 제공

- `\d`: 숫자 (`[0-9]`와 동일)
- `\D`: 숫자가 아닌 문자 (`[^0-9]`와 동일)
- `\w`: 문자와 숫자 및 밑줄(_) (`[a-zA-Z0-9_]`와 동일)
- `\W`: 문자와 숫자 및 밑줄이 아닌 문자
- `\s`: 화이트스페이스 (공백, 탭 `\t`, 줄바꿈 `\n` 등)
- `\b`: 단어 경계 (Word boundary)

3) Raw String(r'...')의 필요성

- 파이썬 문자열에서도 역슬래시는 이스케이프 문자(\n, \t, \b 등)로 먼저 해석되어, 정규식 엔진에 의도와 다른 값이 전달될 수 있음
- 예: "\b"는 파이썬에서 백스페이스 문자로 해석되므로, 단어 경계로 쓰려면 "\\b" 또는 r"\b"로 작성해야 함
- 문자열 앞에 r을 붙이면(Raw String) 역슬래시가 그대로 정규식 엔진에 전달되어 \\를 중복 작성할 필요가 없음
- Raw String 없이 "\d"처럼 쓰면 경고가 발생하므로, 정규식은 항상 r'...'로 작성하는 것이 원칙

```
import re
# 일반 문자열: "\b"가 백스페이스로 해석되어 의도와 다르게 동작
re.search("\bcat\b", "a cat sat")        # None
# 방법 1: 역슬래시를 중복 작성
re.search("\\bcat\\b", "a cat sat")     # 매치됨

# 방법 2: Raw String 사용 (권장)
re.search(r"\bcat\b", "a cat sat")      # 매치됨
```

4) 탐욕적(Greedy) vs 비탐욕적(Non-greedy) 매칭

- *, +는 기본적으로 가능한 한 길게 매치(탐욕적), 뒤에 ?를 붙이면 가능한 한 짧게 매치(비탐욕적)
- 태그, 따옴표 등 구간을 추출할 때 자주 혼동되는 부분

```
s = "<b>a</b><b>b</b>"


# '<b>a</b><b>b</b>'   (탐욕적: 가장 길게)
re.search(r"<.*>", s).group()

# '<b>'                                    (비탐욕적: 가장 짧게)
re.search(r"<.*?>", s).group()
```

4) 예제

실무 데이터 검증 및 마스킹 작업에서 빈번하게 사용되는 정규식 패턴 모음

데이터 유형	정규표현식 패턴	설명 및 주요 메타문자
한국 전화번호	\d{2,3}-\d{3,4}-\d{4}	\d(숫자), {n,m}(반복 횟수)를 활용한 하이픈 포함 전화번호 매칭
이메일 주소	[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}	아이디, @, 도메인, 최상위 도메인(.com, .kr 등) 검증
주민등록번호	\d{6}-[1-4]\d{6}	생년월일 6자리 뒤에 성별 식별자(1~4)와 6자리 숫자가 오는 구조 검증
URL 주소	https?://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}	http 또는 https로 시작하는 웹 주소 패턴 매칭

4. 예제를 통한 핵심 활용법

< 고객 문의 텍스트에서 연락처 추출 · 검증 · 마스킹 >

1) 활용 시나리오

- 고객 문의 목록(텍스트)에서 전화번호와 이메일을 찾아내고, 형식이 올바른지 검증한 뒤, 개인정보를 가려(마스킹) 저장하는 과정

- 앞서 정리한 re.compile → 검색 → Match 객체 조회 → 치환의 흐름을 한 번에 적용

2) 통합 예제 코드

```
import re

text = """고객 문의 목록
홍길동 / 010-1234-5678 / hong@example.com
김영희 / 02-345-6789 / kim.yh@school.ac.kr
잘못된 입력: 010-12-34 / abc@ """

# [1단계] 패턴 컴파일: 반복 사용할 패턴은 미리 컴파일, VERBOSE로 가독성 확보
phone_pat = re.compile(r"""
 (\d{2,3}) # 앞자리 (지역번호/통신사)
 -(\d{3,4}) # 가운데 자리
 -(\d{4}) # 끝 4자리
""", re.VERBOSE)

email_pat = re.compile(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}")

# [2단계] 검색: findall()로 매치된 모든 문자열을 리스트로 추출
print(email_pat.findall(text))

# ['hong@example.com', 'kim.yh@school.ac.kr'] ← 'abc@'는 형식 불일치로 제외


# [3단계] 위치 확인: finditer() + Match 객체의 group(), span()
for m in phone_pat.finditer(text):
 print(m.group(), m.span())
# 010-1234-5678 (14, 27)
# 02-345-6789 (47, 58) ← '010-12-34'는 자릿수 불일치로 제외
```

```
# [4단계] 검증: match()는 문자열 처음부터 검사 → Match 객체 또는 None

print(bool(phone_pat.match("010-1234-5678"))) # True

print(bool(phone_pat.match("전화: 010-1234-5678"))) # False (시작이 다름)
```

```
# [5단계] 마스킹: sub()로 치환 (그룹 번호 \1, \3 재사용 / 치환 함수 사용)
masked = phone_pat.sub(r"\1-****-\3", text)
masked = email_pat.sub(
        lambda m: m.group()[0] + "***" + m.group()[m.group().index("@"):],
                masked
)
print(masked)
# 홍길동 / 010-****-5678 / h***@example.com
# 김영희 / 02-****-6789 / k***@school.ac.kr
```

3) 단계별 핵심 정리

단계	사용 기능	역할
컴파일	re.compile(), re.VERBOSE	패턴 객체 재사용, 주석으로 가독성 향상
추출	findall()	매치되는 문자열 전체를 리스트로 수집
위치 확인	finditer(), group(), span()	매치 내용과 위치를 하나씩 조회
검증	match(), fullmatch()	입력값이 형식에 맞는지 True/False 판단
치환	sub() + 그룹/함수	개인정보 마스킹, 형식 변환

5. 결론

- 정규표현식은 문자열의 검색·검증·치환을 간결한 패턴으로 처리하는 도구
- re 모듈의 compile, findall, match, sub 등을 목적에 맞게 선택하여 사용
- Raw String 사용과 탐욕적 매치에 대한 이해가 정확한 패턴 작성의 핵심